OPEN SOURCE · v0.5

Contextual Memory Architecture (CMA)

A memory layer your agent carries with it.

The closed loop.

Three sentences. The entire thesis.

→ Memory becomes context.

→ Context shapes action.

→ **Action becomes memory.**

// the loop CMA implements

Stateless agents don't compound.

Every session starts cold. Experience never transfers.

01

Forget prior decisions

Every session re-litigates the same architectural choices.

02

Re-learn project context

Conventions, constraints, and quirks rediscovered on every task.

03

Repeat the same mistakes

Postmortems exist but never surface when the same failure recurs.

04

Lose relational context

RAG retrieves chunks, not the decisions linked to the project.

RAG is not memory.

The most useful context is structurally nearest, not semantically nearest.

STANDARD RAG RETRIEVES

- the nearest vector chunk
- by cosine similarity
- ignores edges, time, status
- indifferent to provenance

AGENT MEMORY NEEDS

→ decisions linked to projects

→ postmortems linked to failures

→ patterns inferred across tasks

→ supersession-aware exclusion

Three intelligent functions.

Reasoner frames. Retriever recalls. Recorder forms.

01

REASONER

frames the goal,
decides what context matters

→

02

RETRIEVER

memory → context spec
via hybrid + graph search

→

03

RECORDER

experience → structured
durable memory

// memory feeds the next task

The vault is the memory.

Just markdown. Open in Obsidian. Version with git. Grep from the shell.

async-capital-call-processing.md

```
---
type: decision
title: Async Capital Call Processing
status: accepted
confidence: 0.86
tags: [capital-calls, performance, backend]
---
# Async Capital Call Processing

We decided to move capital call processing
into an async queue.

Uses [[Queue Retry Pattern]].
```

YAML frontmatter

machine-readable metadata

[[Wikilinks]]

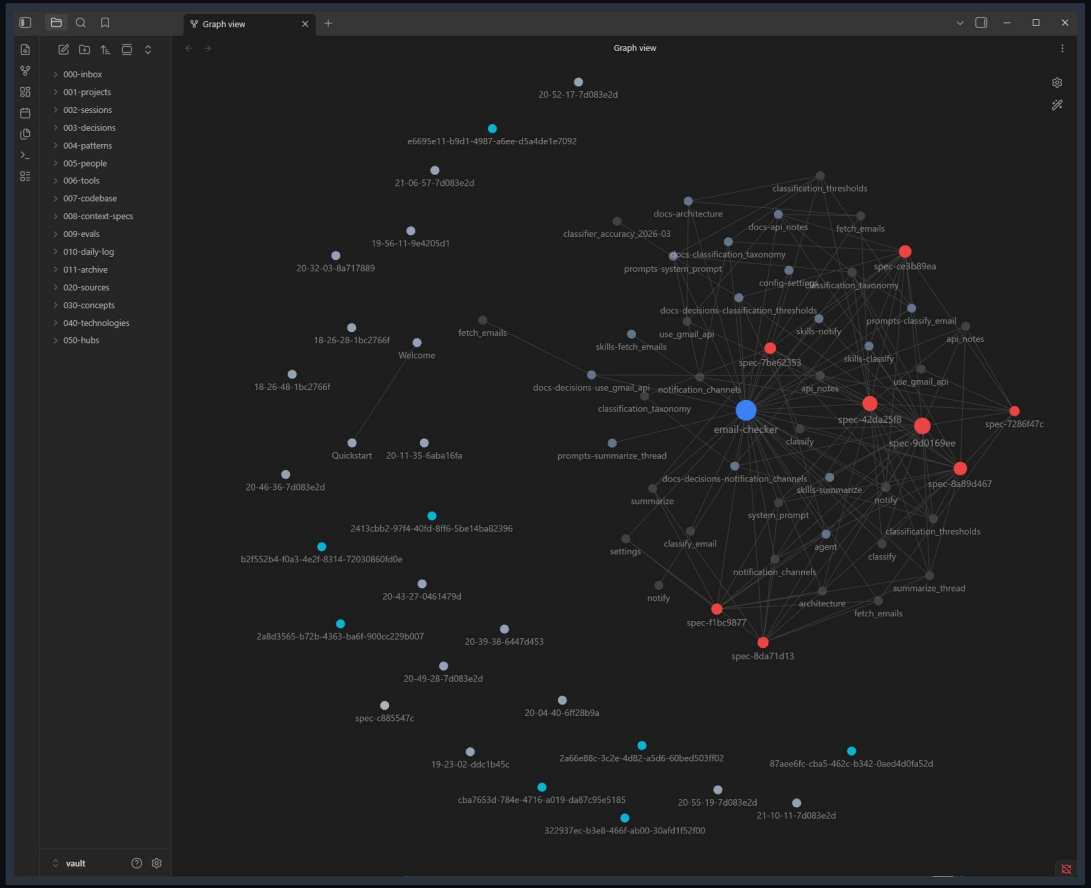
graph edges between notes

Filename

stable record id

Live from a working vault.

An email-checker agent. Every dot, a memory; every line, a relationship.



Obsidian graph view · email-checker agent · spec → source → backlink

Query to context, in five stages.

Deterministic. Inspectable. Reproducible.

01 **Hybrid seed search**

BM25L lexical + optional dense embeddings = hybrid score

02 **Beam-pruned multi-hop traversal**

Expand outward over wikilinks, beam-prune at each depth

03 **Final scoring**

metadata_boost × depth_decay ^ depth

04 **Paragraph-level fragment extract**

Score paragraphs against query, dedupe across nodes

05 **Context Spec assembly**

Structured artifact: fragments + relationship map + provenance

Hybrid scoring, depth-decayed.

Lexical and semantic combined, then modulated by metadata and depth.

```
node_score = α · semantic + (1 − α) · lexical
final_score = node_score · metadata_boost · decay ^ depth
```

DEFAULTS

α	0.7	decay	0.80
beam_width	5	max_depth	2

METADATA BOOSTS

accepted decision	+0.10
superseded decision	−0.50
human_verified	+0.10
confidence ≥ 0.85	+0.08

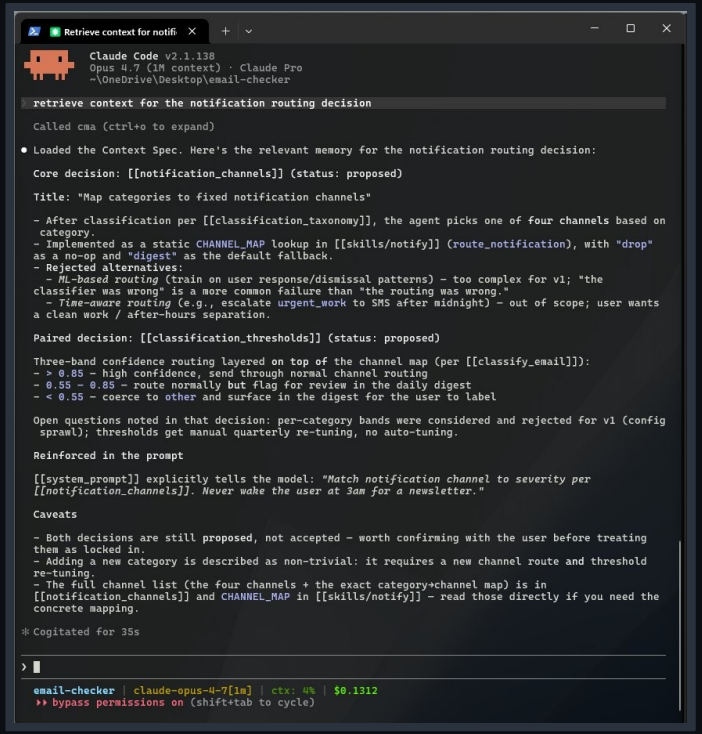
Confidence-gated memory writes.

Low-signal noise never reaches the vault.

STATUS	CONFIDENCE	ACTION	DESTINATION
accepted	any	WRITE	vault/003-decisions/
rejected	any	WRITE	vault/003-decisions/
proposed	≥ 0.75	WRITE	vault/003-decisions/
proposed	0.50 – 0.74	DRAFT	vault/003-decisions/ status=draft
proposed	0.25 – 0.49	PROPOSE	recorder/memory_write_proposals/
any	< 0.25	SKIP	–

What an agent sees.

A retrieve() call surfaces the relevant decision — linked, contextualized, ready to reason over.



OBSERVATION

Structured retrieval

Claude calls cma, gets a Context Spec — not a chunk dump.

Decisions surface as decisions

[[notification_channels]] arrives with status, rationale, and rejected alternatives.

Caveats included

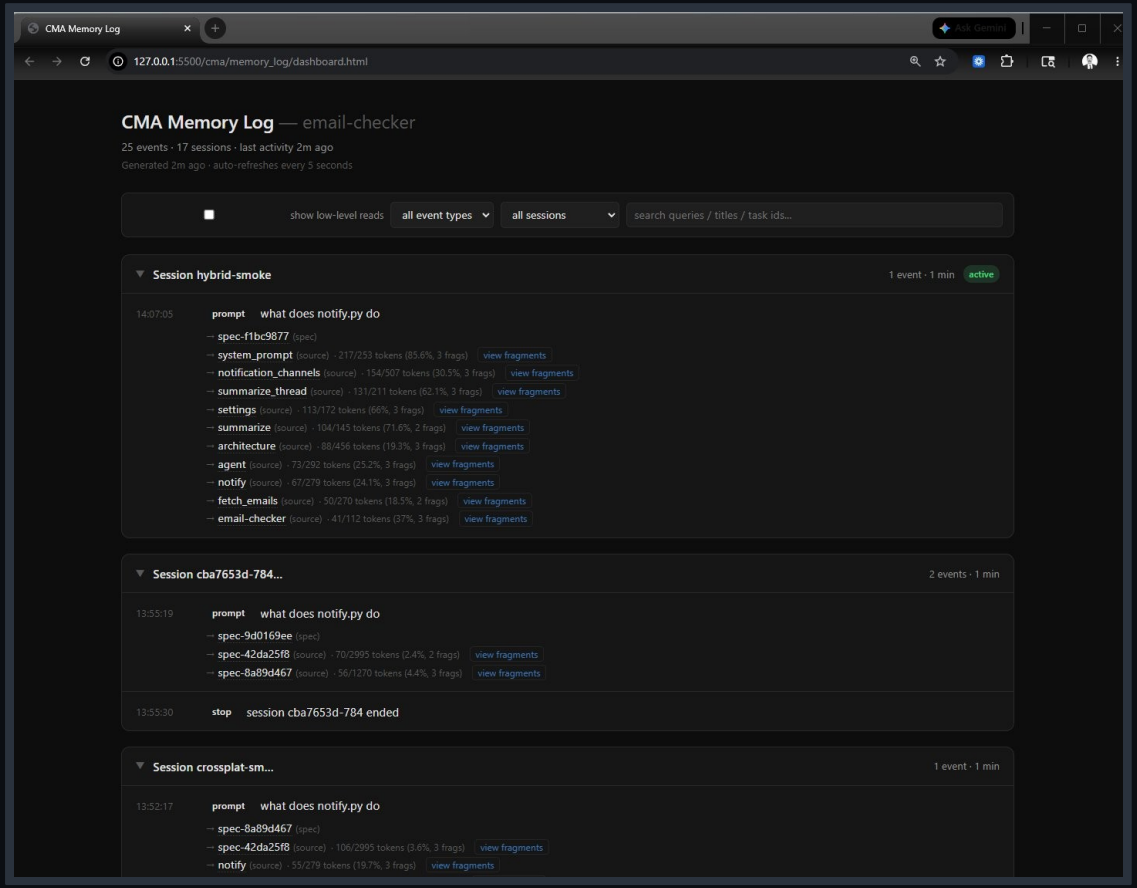
"Both decisions are still proposed, not accepted" — the agent knows what's tentative.

Self-narrating

What was loaded, why it was relevant, what to read next.

Memory you can watch.

Every retrieve, search, and write — logged, grouped by session, linked to every artifact.



cma/memory_log/dashboard.html · static, self-refreshing, no server

Ten MCP tools. Any agent.

Six graph primitives, four orchestrators. Stdio transport.

GRAPH PRIMITIVES

```
search_notes(query, top_k)
get_note(title)
get_outgoing_links(title)
get_backlinks(title)
traverse_graph(start, depth)
search_by_frontmatter(key, value)
```

ORCHESTRATORS

```
retrieve(query, max_depth, beam_width)
record_completion(yaml, dry_run)
graph_health()
reindex()
```

// wires into Claude Code via .mcp.json

Scales to 100K notes on commodity hardware.

Single-CPU NumPy. No ANN required below the 100K threshold.

VAULT SIZE	MARKDOWN	EMBEDDINGS	RAM	QUERY LATENCY
1K notes	~10 MB	~1.5 MB	~50 MB	< 50 ms
10K notes	~100 MB	~15 MB	~250 MB	< 100 ms
100K notes	~1 GB	~150 MB	~1.5 GB	~500 ms
1M notes	~10 GB	~1.5 GB	~10+ GB	**needs ANN**

// beyond ~100K, swap to FAISS or hnswlib. beyond ~500K, shard by domain.

Get started.

Local-first. Open source. Drop-in for any MCP-aware agent.

zsh

```
$ pip install contextual-memory-architecture[all]
$ cma init my-agent
$ cma index my-agent
$ cma mcp serve --project my-agent
```

→ **github.com/danny-watkins/contextual-memory-architecture**

Open source · Local-first · Built for Claude Code · Works with any MCP-aware agent